Intermediate (make search conditional) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `search_tool_integration_practice.py`: this is the exact conditional edge the Build Task needs at its very first step — decide, before anything else runs, whether this question needs the documents at all. **If not:** the Build Task would search on every question, paying for an embedding call and padding the prompt even for "what is 2 + 2?".

Uses the same `practice/` setup as the Basic exercise (Doc08's files, `docs/` and `kb.py`).

Basic Version

Approach 1 — a keyword check

```
# search_tool_integration_practice.py — Intermediate section
from typing import TypedDict

from langchain_openai import ChatOpenAI
from langgraph.graph import END, START, StateGraph

from kb import collection
from retriever import retrieve

model = ChatOpenAI(model="gpt-4o-mini", temperature=0)


class QAState(TypedDict):
    task: str
    sources: list
    answer: str


def search_node(state):
    return {"sources": retrieve(collection, state["task"])}


def answer_node(state):
```

```
    if not state["sources"]:
        return {"answer": model.invoke(state["task"]).content}
    pieces = []
    for source in state["sources"]:
        pieces.append(source["text"])
    context = "\n\n".join(pieces)
    prompt = "Context:\n" + context + "\n\nQuestion: " + state["task"]
    return {"answer": model.invoke(prompt).content}


def should_search(state):
    task = state["task"].lower()
    keywords = ["coffee", "brew", "grind", "water", "beans", "according to"]
    has_match = False
    for keyword in keywords:
        if keyword in task:
            has_match = True
    if has_match:
        return "search"
    return "skip"


def build_graph(router):
    builder = StateGraph(QAState)
    builder.add_node("search_node", search_node)
    builder.add_node("answer_node", answer_node)
    builder.add_conditional_edges(
        START, router, {"search": "search_node", "skip": "answer_node"}
    )
    builder.add_edge("search_node", "answer_node")
    builder.add_edge("answer_node", END)
    return builder.compile()


graph = build_graph(should_search)
for task in ["How hot should the water be when brewing?", "What is 2 + 2?"]:
    result = graph.invoke({"task": task, "sources": [], "answer": ""})
    print(task, "-> searched:", len(result["sources"]) > 0)
```

Expected output:

```
How hot should the water be when brewing? -> searched: True
What is 2 + 2? -> searched: False
```

`add_conditional_edges(START, should_search, {...})` puts the decision at the very start of the graph: before any node runs, `should_search` picks `search_node` or `answer_node`.

`build_graph(router)` takes the routing function as a parameter (Doc08's `chunk_fn` idea), so the approaches below can swap only the router.

Approach 2 — asking a small model to decide

Replace `should_search` with this; the rest stays the same.

```
# search_tool_integration_practice.py — Intermediate section
def should_search(state):
    prompt = (
        "Does answering this question require looking up information "
        "in a set of coffee documents? Answer only yes or no.\n\n"
        "Question: " + state["task"]
    )
    answer = model.invoke(prompt).content.strip().lower()
    if "yes" in answer:
        return "search"
    return "skip"
```

Both work. Approach 1 is free and instant but easy to fool with unexpected wording. Approach 2 catches more phrasings but costs one extra model call on every task.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — a keyword check, with type hints

```
# search_tool_integration_practice.py — Intermediate section
from typing import Callable, TypedDict

from langchain_openai import ChatOpenAI
from langgraph.graph import END, START, StateGraph

from kb import collection
from retriever import retrieve

model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
KEYWORDS = ["coffee", "brew", "grind", "water", "beans", "according to"]


class QAState(TypedDict):
    task: str
    sources: list[dict]
    answer: str


def search_node(state: QAState) -> dict:
    return {"sources": retrieve(collection, state["task"])}


def answer_node(state: QAState) -> dict:
    if not state["sources"]:
        return {"answer": model.invoke(state["task"]).content}
    pieces: list[str] = []
    for source in state["sources"]:
        pieces.append(source["text"])
    context = "\n\n".join(pieces)
    prompt = f"Context:\n{context}\n\nQuestion: {state['task']}"
    return {"answer": model.invoke(prompt).content}


def should_search(state: QAState) -> str:
    task = state["task"].lower()
    has_match = False
```

```
    for keyword in KEYWORDS:
        if keyword in task:
            has_match = True
    if has_match:
        return "search"
    return "skip"


def build_graph(router: Callable[[QAState], str]):
    builder = StateGraph(QAState)
    builder.add_node("search_node", search_node)
    builder.add_node("answer_node", answer_node)
    builder.add_conditional_edges(
        START, router, {"search": "search_node", "skip": "answer_node"}
    )
    builder.add_edge("search_node", "answer_node")
    builder.add_edge("answer_node", END)
    return builder.compile()


def run(graph, task: str) -> dict:
    return graph.invoke({"task": task, "sources": [], "answer": ""})


if __name__ == "__main__":
    graph = build_graph(should_search)
    for task in ["How hot should the water be when brewing?", "What is 2 + 2?"]:
        result = run(graph, task)
        print(f"{task!r} -> searched: {len(result['sources']) > 0}")
```

Expected output:

```
'How hot should the water be when brewing?' -> searched: True
'What is 2 + 2?' -> searched: False
```

Approach 2 — a small classification call

Add this below Approach 1's code (it replaces `should_search`):

```
# search_tool_integration_practice.py — Intermediate section
from langchain_core.messages import HumanMessage

CLASSIFY_PROMPT = (
    "Does answering this question require looking up information "
    "in a set of coffee documents? Answer with exactly one word: "
```

```
    "yes or no.\n\nQuestion: {task}"
)


def should_search(state: QAState) -> str:
    prompt = CLASSIFY_PROMPT.format(task=state["task"])
    answer = model.invoke([HumanMessage(content=prompt)])
    if "yes" in answer.content.strip().lower():
        return "search"
    return "skip"
```

`HumanMessage(content=...)` is LangChain's object for one user message — the same thing as `{"role": "user", "content": ...}`. `CLASSIFY_PROMPT.format(task=...)` fills the `{task}` slot.

Difference from Basic: full type hints, `build_graph()` / `run()` as reusable functions, and a prompt that asks for "exactly one word", which makes the `"yes" in ...` check more reliable. Neither version mixes the two strategies yet — that's Approach 3.

Approach 3 — a hybrid: cheap check first, model call only when ambiguous

Story: keywords miss unexpected phrasings; a model call costs a round trip on every task, even the obvious ones. **If not:** the Build Task's router would either misroute real questions or pay for a model call on every single one.

```
# search_tool_integration_practice.py — Intermediate section
from langchain_core.messages import HumanMessage


def has_keyword(task: str) -> bool:
    lowered = task.lower()
    for keyword in KEYWORDS:
        if keyword in lowered:
            return True
    return False


def ask_model_to_classify(task: str) -> str:
    prompt = CLASSIFY_PROMPT.format(task=task)
    answer = model.invoke([HumanMessage(content=prompt)])
    if "yes" in answer.content.strip().lower():
        return "search"
    return "skip"


def should_search(state: QAState) -> str:
```

```
    task = state["task"]
    # confident hit: a clear keyword match
    if has_keyword(task):
        return "search"
    # confident skip: short, no keyword, no question mark
    if len(task.split()) <= 6 and "?" not in task:
        return "skip"
    # ambiguous: pay for the more accurate check only now
    return ask_model_to_classify(task)
```

Run it with three kinds of question:

```
graph = build_graph(should_search)
tasks = [
    "How hot should the water be when brewing?",
    "thanks, that's all",
    "How long can I store an opened bag before it goes stale?",
]
for task in tasks:
    result = run(graph, task)
    print(f"{task!r} -> searched: {len(result['sources']) > 0}")
```

Expected output:

```
'How hot should the water be when brewing?' -> searched: True
"thanks, that's all" -> searched: False
'How long can I store an opened bag before it goes stale?' -> searched: True
```

Only the third question pays for a model call: it has no keyword, but it is a real question.

Approach 4 — logging every routing decision, for later review

Story: a hybrid router is still a guess, and guesses are worth checking. This keeps Approach 3's logic, but records which branch fired and why. **If not:** "which questions get misrouted?" could only be answered by guessing, never by evidence.

```
# search_tool_integration_practice.py — Intermediate section
from logging_setup import get_logger


def should_search(state: QAState) -> str:
    logger = get_logger("routing")
    task = state["task"]
    if has_keyword(task):
```

```
        logger.info(f"routed=search reason=keyword task={task!r}")
        return "search"
    if len(task.split()) <= 6 and "?" not in task:
        logger.info(f"routed=skip reason=conversational task={task!r}")
        return "skip"
    decision = ask_model_to_classify(task)
    logger.info(f"routed={decision} reason=model_call task={task!r}")
    return decision
```

Expected log lines (with `LOG_LEVEL=INFO`), for the same three tasks:

```
... routing INFO routed=search reason=keyword task='How hot should ...'
... routing INFO routed=skip reason=conversational task="thanks, ..."
... routing INFO routed=search reason=model_call task='How long can ...'
```

(Shortened to fit the page.) The routing decisions don't change — this is purely a record, and reading it later is how you find real misroutes.

Approach 5 — the same entry decision with `set_conditional_entry_point`

Story: LangGraph has a shortcut for "decide at the very start": `builder.set_conditional_entry_point(router, {...})`. It does exactly what `add_conditional_edges(START, router, {...})` does. **If not:** the Build Task's `graph.py`, which uses the shortcut, would be the first time you met it.

```
# search_tool_integration_practice.py — Intermediate section
def build_graph_with_entry_point(router: Callable[[QAState], str]):
    builder = StateGraph(QAState)
    builder.add_node("search_node", search_node)
    builder.add_node("answer_node", answer_node)
    # the same as add_conditional_edges(START, router, {...})
    builder.set_conditional_entry_point(
        router, {"search": "search_node", "skip": "answer_node"}
    )
    builder.add_edge("search_node", "answer_node")
    builder.add_edge("answer_node", END)
    return builder.compile()


graph = build_graph_with_entry_point(should_search)
for task in ["How hot should the water be when brewing?", "What is 2 + 2?"]:
    result = run(graph, task)
    print(f"{task!r} -> searched: {len(result['sources']) > 0}")
```

Expected output: the same two lines as Approach 1.

Difference from Approach 1, and between Approaches 3–5: Approaches 1/2 commit to one strategy for every task. Approach 3 pays for the model call only on the ambiguous middle. Approach 4 adds a record of *why* each decision was made. Approach 5 changes nothing about the decision — it's the shorter way to write the same entry edge.

Which one should you actually write? Start with Approach 1's keywords — free, instant, and good enough while you build the rest. Move to Approach 3 once you've seen keywords misroute a real question. Add Approach 4's logging once real, varied users send questions. Use whichever of the two entry-edge spellings (Approach 1 or 5) you find clearer — they're the same.